

Kubernetes Interview Question & Answer Guide

Comprehensive answers covering Fundamentals, Architecture, EKS,
CI/CD, Security, Monitoring & Troubleshooting

Target Role	DevOps / Cloud / SRE Engineer
Focus Areas	EKS, Docker, Terraform, Jenkins, GitHub Actions
Monitoring Stack	Prometheus, Grafana, ELK, Trivy

1. Kubernetes Fundamentals

Q: What is Kubernetes?

Kubernetes (K8s) is an open-source container orchestration platform that automates the deployment, scaling, and management of containerized applications across a cluster of machines.

Q: Why do organizations use Kubernetes?

- Automates deployment, scaling, and healing of containers
- Handles load balancing and traffic distribution
- Enables zero-downtime rolling updates and rollbacks
- Efficiently manages resources across multiple servers
- Works across on-premises, cloud, and hybrid environments

Q: What problems does Kubernetes solve?

- Manual container management becomes impossible at scale — K8s automates it
- Containers crash unpredictably — K8s auto-restarts them (self-healing)
- Traffic spikes require more instances — K8s auto-scales
- Deploying without downtime is hard — K8s handles rolling updates
- Managing config and secrets across environments is complex — K8s has built-in solutions

Q: What are the main components of Kubernetes?

- Control Plane: API Server, etcd, Scheduler, Controller Manager
- Worker Nodes: kubelet, kube-proxy, container runtime
- Core Objects: Pods, Deployments, Services, ConfigMaps, Secrets

Q: What is a Kubernetes cluster?

A Kubernetes cluster is a set of machines (nodes) that run containerized applications. It consists of at least one control plane node (manages the cluster) and one or more worker nodes (run the actual workloads).

Q: What is the difference between Docker and Kubernetes?

- Docker: A tool to build and run individual containers on a single machine
- Kubernetes: A platform to orchestrate and manage many containers across many machines
- Think of Docker as the shipping container; Kubernetes as the entire port managing thousands of containers

2. Kubernetes Architecture

Q: What are the components of the control plane?

- kube-apiserver: The front door — all communication goes through it
- etcd: Key-value store that holds all cluster state and configuration
- kube-scheduler: Decides which worker node a new Pod should run on
- kube-controller-manager: Runs controllers that ensure desired state matches actual state

Q: What is kube-apiserver?

The kube-apiserver is the central management component. Every action in Kubernetes (kubectl commands, internal component communication, UI calls) goes through the API server. It validates requests, authenticates users, and updates etcd.

Q: What is etcd?

etcd is a distributed key-value store that acts as Kubernetes' brain/memory. It stores the entire cluster state — all configurations, Pod definitions, Secrets, etc. If etcd is lost without backup, the cluster state is gone. This is why backing up etcd is critical.

Q: What is kube-scheduler?

The scheduler watches for newly created Pods that have no assigned node, then selects the best node for them based on resource availability, affinity rules, taints/tolerations, and other constraints.

Q: What is kubelet?

kubelet is an agent running on every worker node. It receives Pod specifications from the API server and ensures the containers described in those specs are running and healthy. It's the node's 'manager'.

Q: What is kube-proxy?

kube-proxy runs on every node and handles network routing. It maintains network rules (iptables/IPVS) that allow Services to route traffic to the correct Pods, enabling communication between services inside and outside the cluster.

Q: What happens if the API server fails?

- Existing workloads continue running — Pods don't stop immediately
- No new Pods can be scheduled or created
- kubectl commands stop working
- Auto-healing (controllers) stops functioning until API server recovers
- In EKS, AWS manages the control plane and ensures high availability

3. Pods

Q: What is a Pod?

A Pod is the smallest deployable unit in Kubernetes. It wraps one or more containers that share the same network namespace (same IP address) and storage volumes. Containers in a Pod communicate via localhost.

Q: When would you use multiple containers in a Pod?

- Sidecar pattern: A logging container ships logs from the main app container
- Ambassador pattern: A proxy container handles external communication
- Adapter pattern: A container transforms the main container's output

Rule of thumb: containers belong in the same Pod only if they must share network/storage and lifecycle.

Q: What is an Init Container?

An init container runs before the main application container starts. It completes a setup task (like waiting for a database to be ready, or loading configuration) and must finish successfully before the main container starts.

Q: What is a Sidecar Container?

A sidecar runs alongside the main container in the same Pod, enhancing or extending its functionality. Common examples: log shippers (Fluent Bit), service mesh proxies (Envoy/Istio), or secret injectors (Vault Agent).

Q: What are Pod phases?

- Pending: Pod accepted but containers not yet running (scheduling or image pull in progress)
- Running: At least one container is running
- Succeeded: All containers exited with code 0
- Failed: At least one container exited with non-zero code
- Unknown: Pod state cannot be determined

Q: SCENARIO: Pod is stuck in Pending state — how do you troubleshoot?

Step 1: Run `kubectl describe pod <pod-name>` and check the Events section.

- 'Insufficient CPU/memory' → Node doesn't have enough resources; check node capacity
- 'No nodes available' → All nodes may have taints; check tolerations
- 'PVC not bound' → Persistent Volume Claim is pending; check storage
- 'Image pull error' → Wrong image name or missing imagePullSecret

```
kubectl describe pod <pod-name>
kubectl get events --sort-by='.lastTimestamp'
```

Q: SCENARIO: Pod is in CrashLoopBackOff — how do you troubleshoot?

The container is crashing repeatedly. Kubernetes waits longer between each restart attempt.

- Check logs: `kubectl logs <pod-name> --previous` (see logs from last crash)
- Common causes: app startup error, missing env variable, wrong command, failed health check
- Check resource limits — OOMKilled means memory limit too low
- Use `kubectl describe` to check exit codes and events

```
kubectl logs <pod-name> --previous
kubectl describe pod <pod-name>
```

4. Deployments

Q: What is a Deployment?

A Deployment is a Kubernetes object that manages a set of identical Pods. You declare the desired state (e.g., '3 replicas of my-app:v2') and the Deployment controller ensures the cluster matches that state at all times — handling rollouts, rollbacks, and self-healing.

Q: What is a ReplicaSet?

A ReplicaSet ensures a specified number of Pod replicas are always running. A Deployment creates and manages ReplicaSets under the hood. You typically never create ReplicaSets directly — you use Deployments.

Q: What is a rolling update?

A rolling update gradually replaces old Pods with new ones, ensuring the application stays available throughout the update. Kubernetes brings up new Pods before terminating old ones, based on `maxSurge` and `maxUnavailable` settings.

Q: What is maxUnavailable and maxSurge?

- `maxUnavailable`: Maximum number of Pods that can be unavailable during update (default: 25%)
- `maxSurge`: Maximum number of extra Pods that can exist above desired count (default: 25%)

Example: 4 replicas, maxUnavailable=1, maxSurge=1 → at most 1 Pod down, at most 5 Pods total during rollout.

Q: How do you rollback a Deployment?

```
kubectl rollout undo deployment/<name>
kubectl rollout undo deployment/<name> --to-revision=2
kubectl rollout history deployment/<name>
```

Q: How do you scale a Deployment?

```
kubectl scale deployment <name> --replicas=5
# Or edit the YAML and kubectl apply
```

5. Services

Q: Why are Services required?

Pods are temporary — they get new IP addresses every time they restart. Services provide a stable DNS name and IP address that always points to the correct Pods, regardless of Pod restarts or replacements.

Q: What are the different Service types?

- **ClusterIP (default):** Internal-only. Accessible only within the cluster. Used for service-to-service communication.
- **NodePort:** Exposes the service on a static port on every node's IP. Accessible externally via `<NodeIP>:<NodePort>`.
- **LoadBalancer:** Creates an external cloud load balancer (e.g., AWS ELB). Best for production external access.
- **ExternalName:** Maps the service to an external DNS name. Used to access external services by internal name.

Q: How does traffic reach a Pod?

External request → LoadBalancer/Ingress → Service (selects Pods by label) → kube-proxy routes to Pod IP → Container

Q: SCENARIO: Application is running but inaccessible — what do you check?

1. Verify Pod is Running: **kubectl get pods**
2. Check Service exists and has correct selectors: **kubectl describe svc <name>**
3. Verify Endpoints are populated: **kubectl get endpoints <svc-name>** — if empty, label mismatch
4. Check Service port vs container port match
5. Check Network Policies — may be blocking traffic
6. For Ingress: check Ingress rules and Ingress Controller logs

6. Ingress

Q: What is Ingress?

Ingress is a Kubernetes API object that manages external HTTP/HTTPS access to services in the cluster. It provides URL-based routing, SSL termination, and name-based virtual hosting — all through a single entry point.

Q: Why use Ingress instead of LoadBalancer?

- One LoadBalancer (expensive) can serve many services via path/host-based routing
- Central SSL termination — manage certificates in one place
- Supports redirects, rewrites, and rate limiting
- Example: `/app1` → service-a, `/app2` → service-b, all via one LB

Q: What is an Ingress Controller?

The Ingress Controller is the actual implementation that fulfills Ingress rules. Popular options: nginx Ingress Controller, AWS Load Balancer Controller (ALB), Traefik. Without an Ingress Controller, Ingress resources do nothing.

Q: How does AWS Load Balancer Controller work?

- Runs inside EKS cluster as a Pod
- Watches for Ingress resources with annotation **kubernetes.io/ingress.class: alb**
- Automatically provisions an AWS Application Load Balancer
- Configures target groups to route traffic to the correct Pods
- Supports path-based and host-based routing natively

Q: What is path-based vs host-based routing?

- **Path-based:** app.com/api → service-api, app.com/web → service-web (same domain, different paths)
- **Host-based:** api.app.com → service-api, web.app.com → service-web (different subdomains)

7. ConfigMaps and Secrets

Q: What is a ConfigMap?

A ConfigMap stores non-sensitive configuration data as key-value pairs. It decouples configuration from container images, allowing the same image to run with different configs in dev/staging/prod.

Q: What is a Secret?

A Secret stores sensitive data (passwords, tokens, keys) in base64-encoded format. Unlike ConfigMaps, Secrets are intended for sensitive information and can be encrypted at rest.

Q: Are Kubernetes Secrets really secure?

By default, Secrets are only base64-encoded — NOT encrypted. Anyone with cluster access can decode them. To properly secure Secrets:

- Enable encryption at rest in etcd
- Use RBAC to restrict who can read Secrets
- Use external secret managers: AWS Secrets Manager, HashiCorp Vault, or External Secrets Operator
- Avoid logging Secret values in application logs

Q: How do you inject ConfigMaps/Secrets into Pods?

- **Environment variables:** envFrom or env with secretKeyRef/configMapKeyRef
- **Volume mounts:** Mount as files inside the container filesystem
- Volume-mounted Secrets update automatically when the Secret is changed; env vars don't

Q: How do you update ConfigMaps without restarting Pods?

If mounted as a volume, Kubernetes automatically propagates updates within ~1 minute. For env var injection, you must restart the Pod to pick up changes.

8. Storage

Q: What is a Persistent Volume (PV)?

A PV is a piece of storage in the cluster provisioned by an admin or dynamically. It's a cluster resource independent of any Pod's lifecycle — data survives Pod deletion.

Q: What is a Persistent Volume Claim (PVC)?

A PVC is a request for storage by a user. A Pod uses a PVC to request a specific amount and type of storage. Kubernetes binds the PVC to a suitable PV.

Q: What is StorageClass and dynamic provisioning?

StorageClass defines the type of storage (e.g., AWS gp2 EBS, gp3 SSD). With dynamic provisioning, when a PVC is created, Kubernetes automatically creates a PV using the StorageClass — no manual PV creation needed.

Q: What is ReadWriteOnce?

- **ReadWriteOnce (RWO):** Can be mounted by a single node at a time (typical for EBS)
- **ReadOnlyMany (ROX):** Can be mounted as read-only by many nodes
- **ReadWriteMany (RWX):** Can be mounted read-write by many nodes (EFS, NFS)

Q: How does EBS integrate with EKS?

The EBS CSI Driver (Container Storage Interface) is installed in EKS. When a PVC requests an EBS StorageClass, the CSI driver automatically creates an EBS volume in AWS and attaches it to the correct node.

9. Networking

Q: How does Kubernetes networking work?

- Every Pod gets its own unique IP address
- All Pods can communicate with each other without NAT (flat network)
- Services provide stable IPs/DNS for groups of Pods
- Network is implemented by a CNI (Container Network Interface) plugin

Q: What CNI does EKS use?

EKS uses the AWS VPC CNI plugin. Each Pod gets a real VPC IP address directly from the node's ENI (Elastic Network Interface). This means Pods are first-class citizens in the VPC network and can be accessed like any EC2 resource.

Q: What is CoreDNS?

CoreDNS is the DNS server inside Kubernetes. It resolves service names to their ClusterIP addresses. For example, a Pod can reach a Service named 'my-service' in namespace 'prod' via:
my-service.prod.svc.cluster.local

Q: What are Network Policies?

Network Policies are firewall rules for Pods. By default, all Pods can talk to all other Pods. Network Policies let you restrict traffic — e.g., 'only allow the frontend Pod to talk to the backend Pod on port 8080'.

Q: SCENARIO: Two Pods cannot communicate — how do you troubleshoot?

1. Check both Pods are Running: `kubectl get pods`
2. Check Services exist and have correct endpoints
3. Test with `kubectl exec`: **`kubectl exec -it <pod> -- curl <service-ip>:port`**
4. Check Network Policies — a policy may be blocking traffic
5. Verify same namespace or use FQDN if cross-namespace
6. Check security groups (EKS) — node security group may block Pod-to-Pod traffic

10. Resource Management

Q: What are Requests and Limits?

- **Requests:** The minimum resources guaranteed to the container. Used by scheduler to place Pod on a node.
- **Limits:** The maximum resources the container can use. Container is throttled (CPU) or killed (memory) if exceeded.

Q: What is OOMKilled?

OOMKilled (Out Of Memory Killed) means the container exceeded its memory limit. The Linux kernel killed the process to protect the node. Fix by either increasing the memory limit or finding and fixing a memory leak in the application.

Q: How do CPU limits work?

CPU limits use Linux cgroups to throttle the container. Unlike memory, exceeding CPU limit does NOT kill the Pod — it just slows it down. This can cause latency issues without obvious errors.

Q: How do resource requests affect scheduling?

The scheduler only places a Pod on a node if the node has enough allocatable resources to satisfy the Pod's requests. A node can be 'full' for scheduling purposes while still having unused actual capacity (if pods are using less than they requested).

11. Health Checks (Probes)

Q: What is a Liveness Probe?

A liveness probe checks if the container is alive. If it fails, Kubernetes kills the container and restarts it. Use it to detect deadlocks or hung processes that aren't crashing but aren't working either.

Q: What is a Readiness Probe?

A readiness probe checks if the container is ready to receive traffic. If it fails, the Pod is removed from Service endpoints — traffic stops going to it, but the container is NOT restarted. Use it during startup or temporary unavailability.

Q: Key difference: Liveness vs Readiness?

- **Liveness fail:** Container is killed and restarted (restores function)
- **Readiness fail:** Pod removed from load balancer rotation (no traffic, no restart)
- You often need both: Liveness to recover from crashes, Readiness to prevent traffic during startup

Q: What is a Startup Probe?

A startup probe gives slow-starting containers extra time to initialize before liveness/readiness probes kick in. If startup probe succeeds, liveness and readiness probes take over. Prevents premature killing of apps that take a long time to start.

Q: What probe types exist?

- **HttpGet:** HTTP GET request to a specific path/port — success if 200-399 response
- **tcpSocket:** Checks if a port is open
- **exec:** Runs a command inside the container — success if exit code is 0
- **grpc:** gRPC health check

12. Scaling

Q: What is Horizontal Pod Autoscaler (HPA)?

HPA automatically scales the number of Pod replicas based on observed metrics (CPU, memory, or custom metrics). It queries the Metrics Server and adjusts replica count to keep utilization at the target level.

Q: What is Cluster Autoscaler?

Cluster Autoscaler automatically adjusts the number of nodes in a node group. It adds nodes when Pods are pending due to insufficient resources, and removes nodes when they're underutilized for an extended period.

Q: Difference between HPA and Cluster Autoscaler?

- **HPA:** Scales Pods (horizontal — adds more instances of your app)
- **Cluster Autoscaler:** Scales Nodes (adds/removes EC2 instances/VMs)
- They work together: HPA creates more Pods → Pods can't schedule → CA adds nodes

Q: How does EKS Auto Scaling work?

- HPA scales Pods based on metrics from Prometheus/Metrics Server
- Cluster Autoscaler watches for pending Pods and scales AWS Auto Scaling Groups

- KEDA (Kubernetes Event-driven Autoscaling) can scale based on SQS queue depth, Kafka lag, etc.

13. Kubernetes Security

Q: What is RBAC?

Role-Based Access Control (RBAC) controls who can do what in the cluster. You define Roles (permissions) and bind them to Users, Groups, or Service Accounts via RoleBindings.

Q: Difference between Role and ClusterRole?

- **Role:** Grants permissions within a specific namespace only
- **ClusterRole:** Grants permissions cluster-wide, across all namespaces
- **RoleBinding:** Binds a Role/ClusterRole to a user/SA within a namespace
- **ClusterRoleBinding:** Binds a ClusterRole cluster-wide

Q: What are Service Accounts?

Service Accounts are identities for Pods. When a Pod needs to call the Kubernetes API or access AWS services (via IRSA), it uses its Service Account. Always use dedicated Service Accounts with minimal permissions (principle of least privilege).

Q: What is IRSA?

IAM Roles for Service Accounts (IRSA) allows Pods in EKS to assume IAM roles without needing static AWS credentials. The Pod's Service Account is annotated with an IAM role ARN, and AWS STS provides temporary credentials via the OpenID Connect (OIDC) provider.

Q: SCENARIO: A developer accidentally gets cluster-admin permissions — how do you fix it?

1. Immediately revoke: delete the ClusterRoleBinding giving cluster-admin
2. Audit: check kubectl logs and API server audit logs for actions taken
3. Rotate any Secrets or credentials potentially exposed
4. Create a properly scoped Role with only necessary permissions
5. Implement policy: use OPA/Gatekeeper to prevent accidental cluster-admin grants

Q: How do you scan container images? (Trivy)

- Trivy scans images for known CVEs in OS packages and language libraries
- Integrate in CI pipeline: **trivy image myapp:latest**
- Set severity threshold to block CRITICAL vulnerabilities from deploying
- Also scan Kubernetes manifests: **trivy config ./k8s-manifests/**
- Scan running cluster: **trivy k8s --report summary**

14. Monitoring and Logging

Q: How does Prometheus work in Kubernetes?

- Prometheus scrapes metrics from endpoints (e.g., /metrics) at regular intervals
- ServiceMonitor CRD (from kube-prometheus-stack) tells Prometheus which Services to scrape
- Metrics are stored in a time-series database
- AlertManager handles alerting based on Prometheus rules

Q: What is a ServiceMonitor?

A ServiceMonitor is a custom resource (CRD) used by the Prometheus Operator. It defines which Services Prometheus should scrape for metrics, including the path, port, and interval. It allows adding new scrape targets without editing Prometheus config files.

Q: Key metrics to monitor in Kubernetes?

- **Pod level:** CPU/Memory usage vs limits, restart count, readiness
- **Node level:** CPU, memory, disk pressure, network I/O
- **Cluster level:** API server latency, etcd performance, scheduler queue depth
- **Application level:** Request rate, error rate, latency (RED method)

Q: How does ELK work in Kubernetes?

- **Fluent Bit/Fluentd:** DaemonSet on every node, collects container logs from /var/log/containers/
- **Logstash/Elasticsearch:** Indexes and stores logs
- **Kibana:** Dashboard for searching and visualizing logs
- Logs are enriched with Pod name, namespace, and container labels automatically

Q: How do you investigate a failing Pod?

1. `kubectl get pods` → check status and restart count
2. `kubectl describe pod` → check Events section (scheduling failures, probe failures)
3. `kubectl logs <pod>` → current logs
4. `kubectl logs <pod> --previous` → logs from last crash
5. `kubectl exec -it <pod> -- /bin/sh` → shell into container
6. `kubectl top pod` → check resource usage
7. Check Grafana dashboards and Kibana for historical data

15. Amazon EKS

Q: What is Amazon EKS?

Amazon Elastic Kubernetes Service (EKS) is a managed Kubernetes service. AWS manages the control plane (API server, etcd, scheduler) with SLA guarantees, multi-AZ HA, and automatic upgrades. You manage the worker nodes.

Q: What does AWS manage vs customer in EKS?

- **AWS manages:** kube-apiserver, etcd, kube-controller-manager, kube-scheduler (all control plane components), control plane HA and upgrades
- **Customer manages:** Worker nodes (EC2/Fargate), node OS patching, add-ons (CoreDNS, kube-proxy, VPC CNI), IAM roles, Security Groups, cluster upgrades

Q: How do worker nodes join EKS?

- Nodes run a bootstrap script that calls the EKS API with the cluster name
- The aws-auth ConfigMap in kube-system maps IAM roles to Kubernetes users/groups
- With Managed Node Groups, AWS automates this process
- The node IAM role needs policies: AmazonEKSEKSPolicy, AmazonEKS_CNI_Policy, AmazonEC2ContainerRegistryReadOnly

Q: What is the aws-auth ConfigMap?

aws-auth is a ConfigMap in the kube-system namespace that maps AWS IAM roles/users to Kubernetes RBAC users and groups. It's how EKS authenticates AWS identities into the Kubernetes API. Misconfiguring it can lock you out of the cluster.

Q: What is IRSA and how does it work?

- IRSA = IAM Roles for Service Accounts
- EKS cluster has an OIDC provider registered with AWS IAM
- Annotate a ServiceAccount with `eks.amazonaws.com/role-arn: <role-arn>`
- AWS injects temporary credentials into the Pod via a projected token volume
- Pod can now call AWS APIs (S3, DynamoDB, etc.) without hardcoded credentials

Q: What is Fargate in EKS?

AWS Fargate is a serverless compute option for EKS. Pods run on managed infrastructure — you don't see or manage EC2 nodes. AWS allocates resources per Pod. Good for batch jobs or workloads with unpredictable patterns, but no DaemonSets and limited customization.

Q: Difference between Managed and Self-Managed Node Groups?

- **Managed Node Groups:** AWS automates node provisioning, AMI updates, and draining during upgrades. Easier to manage.
- **Self-Managed:** You manage EC2 ASGs manually. More control but more operational overhead.
- Managed Node Groups are recommended unless you need custom AMIs or specific configurations.

16. CI/CD and Kubernetes

Q: How does Jenkins deploy to EKS?

- Jenkins runs a pipeline that builds the Docker image and pushes it to ECR
- Jenkins uses kubectl (with kubeconfig or AWS credentials) to apply updated manifests
- Or uses Helm: `helm upgrade --install <release> <chart> --set image.tag=$BUILD_NUMBER`
- Jenkins needs an IAM role with EKS access and the cluster's kubeconfig

Q: How does GitHub Actions deploy to EKS?

- Use the **aws-actions/configure-aws-credentials** action to set up IAM credentials
- Use **aws-actions/amazon-eks-kubectl** or run kubectl commands directly
- Store kubeconfig in GitHub Secrets or generate it dynamically using `aws eks update-kubeconfig`
- For security, use OIDC-based authentication (no static AWS keys needed)

Q: What is Helm?

Helm is the package manager for Kubernetes. It bundles Kubernetes manifests into reusable 'Charts' with templating support. Instead of managing dozens of YAML files per environment, you manage a single chart with environment-specific values files.

Q: What are Helm values?

Values are the configuration parameters for a Helm chart. They're defined in `values.yaml` with defaults, and can be overridden per environment. For example: `image.tag`, `replicaCount`, `ingress.host`, `resources.limits.memory`.

Q: How do you rollback a Helm release?

```
helm rollback <release-name> [revision]
helm history <release-name> # See revision history
```

Q: SCENARIO: Pipeline succeeds but deployment fails — how do you investigate?

1. Check deployment rollout: **`kubectl rollout status deployment/<name>`**
2. Check new Pods: `kubectl get pods` — look for `CrashLoopBackOff` or `ImagePullBackOff`
3. `kubectl describe pod` on a failing new Pod — check events
4. `kubectl logs` on the new Pod — application startup error?
5. Compare new image tag — was the right image pushed to ECR?
6. Check if new config/secrets are correctly injected
7. Check resource limits — node may not have capacity for new Pods

17. Troubleshooting Quick Reference

Q: Common error states and causes:

Error State	Common Causes	First Command
-------------	---------------	---------------

Pending	Insufficient resources, taint mismatch, PVC unbound	kubectl describe pod
ContainerCreating	Image pull issue, volume mount failure, CNI problem	kubectl describe pod
CrashLoopBackOff	App error, missing config, wrong command, OOM	kubectl logs --previous
ImagePullBackOff	Wrong image name, missing imagePullSecret, ECR auth	kubectl describe pod
OOMKilled	Memory limit too low, memory leak	kubectl top pod
Terminating (stuck)	Finalizers not cleared, volume detach stuck	kubectl describe pod
Node NotReady	kubelet down, disk pressure, memory pressure	kubectl describe node

Q: Essential kubectl Commands Every Interviewer Expects:

```
kubectl get pods -n -o wide
kubectl describe pod
kubectl logs --previous
kubectl exec -it -- /bin/sh
kubectl top pod / kubectl top node
kubectl get events --sort-by='.lastTimestamp'
kubectl rollout status deployment/
kubectl rollout undo deployment/
kubectl scale deployment --replicas=3
kubectl get svc / kubectl get ingress
kubectl apply -f manifest.yaml
kubectl delete pod # Force reschedule
```

18. Behavioral / Experience-Based Questions

Q: Describe a Kubernetes production issue you faced.

STAR Format — adapt with your real experience:

- **Situation:** During my internship, a deployment to EKS caused a spike in CrashLoopBackOff Pods after a routine update.
- **Task:** Identify and resolve the issue with minimal downtime.
- **Action:** Used kubectl logs --previous to find the root cause — a new environment variable was missing in the updated ConfigMap. Rolled back the deployment immediately using kubectl rollout undo, then updated the ConfigMap with the correct value and re-deployed.
- **Result:** Service restored within 10 minutes. Added a checklist to validate all required environment variables before merging PRs.

Q: Tell me about a deployment failure you resolved.

Frame your answer around: What failed? How did you detect it? What commands/tools did you use? How did you fix it? What did you learn? Use kubectl rollout status, kubectl describe, and kubectl logs as your detective tools in the story.

Q: Have you ever caused downtime? What happened?

Be honest and focus on learning. Interviewers respect honesty + ownership. Structure: what happened → immediate response → root cause → fix → prevention. Example: a wrong resource limit caused OOMKilled → updated limits and added memory alerts in Grafana.

Q: How do you prioritize incidents?

- P1 (production down): Drop everything, immediate response
- P2 (degraded performance): Respond within 15-30 minutes

- P3 (non-critical, workaround exists): Schedule fix
- Always communicate status to stakeholders during incidents
- Use runbooks to speed up resolution for known issue patterns

Key Topics Checklist

■ Topic	Priority
Pods, Deployments, ReplicaSets	Critical
Services: ClusterIP, NodePort, LoadBalancer	Critical
Ingress & AWS ALB Controller	Critical
ConfigMaps & Secrets (+ secure management)	Critical
PV, PVC, StorageClass (EBS/EFS)	High
Resource Requests & Limits (OOMKilled)	Critical
Liveness & Readiness Probes	Critical
HPA & Cluster Autoscaler	High
RBAC, Service Accounts, IRSA	Critical
EKS Architecture (aws-auth, OIDC, Fargate)	Critical
Prometheus, Grafana, ELK setup	High
Terraform + EKS provisioning	High
Jenkins/GitHub Actions + Helm deployments	High
Trivy image scanning in CI/CD	Medium
Troubleshooting: CrashLoop, Pending, ImagePullBackOff	Critical

Good luck with your interviews!